

HapticNV MCU 软件移植指南

1	概述	3
2	驱动适配	4
3	移植有效性验证	5
4	功能和接口	6
5	单马达调用示例	9
6	双马达调用示例	10
7	F0 校准说明	12
8	波形示例	13
9	注意事项	14

1 概述

驱动源文件	haptic_nv.c,haptic_nv.h,haptic_nv_reg.h,haptic_nv_params.c,haptic_nv_config.c, aw862x.c,aw862xx.c,aw8623x.c,aw8624x
芯片支持	AW862X :aw8623 aw8624 AW862XX :aw86214 aw86223 aw86224 aw86225 AW8623X :aw86233 aw86234 aw86235 AW8624X :aw86243 aw86245
I ² C 地址范围	0x5A/0x58

2 驱动适配

1. 根据实际应用平台，在 [haptic_nv_config.c](#) 中，编写对应有关 i2c 读写，延时，定时器以及 GPIO 控制的函数，默认使用参考平台 STM32H743VIT6 来实现
2. 打开驱动包中的 [aw862x.c](#)/[aw862xx.c](#)/[aw8623x.c](#)/[aw8624x.c](#) 文件，根据使用马达的标称 f0 和额定电压修改其中的 dts 结构体中的 f0_pre 和 cont_lra_vrms 参数
3. 把所有*.c、*.h 文件放入工程目录“Awinic”文件夹下
4. 鼠标右键点击“project”，新建“Awinic”模块，添加 [aw862x.c](#), [aw862xx.c](#), [aw8623x.c](#),[aw8624x.c](#),[haptic_nv.c](#), [haptic_nv_config.c](#),[haptic_nv_params.c](#) 文件包含在工程，并添加头文件路径
5. 打开 [haptic_nv.h](#)，根据使用的芯片选择芯片宏控 [AW862X_DRIVER](#)/[AW862XX_DRIVER](#)/[AW8623X_DRIVER](#)/[AW8624X_DRIVER](#)，减少代码量，节约平台资源
6. 打开 [haptic_nv.h](#)，根据是否使用 RTP autosin 功能，打开和关闭 [AW_RTP_AUTOSIN](#) 宏控(默认关闭)
7. 打开 [haptic_nv.h](#)，[AW_IRQ_CONFIG](#) 配置使用中断，否则使用 trig，[AW_CHECK_RAM_DATA](#) 配置 ram 的回读校验，[AW_ENABLE_RTP_PRINT_LOG](#) 打开 rtp 播放过程详细 log，[HAPTIC_NV_DOUBLE](#) 配置双马达使用
8. 在 [main.c](#) 中添加 `#include "haptic_nv.h"`，并配置好 I2C 和定时器(驱动默认使用 hi2c1(hi2c2)和 htim3)，编译工程

3 移植有效性验证

芯片成功识别:

```
[haptic_nv]parse_chipid: detected aw8623.  
[haptic_nv]sw_reset: enter!  
[haptic_nv]aw862x_offset_cal: enter  
[haptic_nv]aw862x_interrupt_setup: reg SYSINT=0x30  
[haptic_nv]aw862x_play_mode: enter standby mode  
[haptic_nv]aw862x_protect_config: enable protection mode
```

```
[haptic_nv][0749]haptic_nv_boot_init: haptic_nv driver version v0.2.0  
[haptic_nv][0091]haptic_nv_read_chipid: reading chip id  
[haptic_nv][0091]haptic_nv_read_chipid: reading chip id  
[haptic_nv][0091]haptic_nv_read_chipid: reading chip id  
[haptic_nv][0091]haptic_nv_read_chipid: reading chip id  
[haptic_nv][0091]haptic_nv_read_chipid: reading chip id  
[haptic_nv][0466]parse_chipid: try to replace i2c addr [(0x58)] to read chip id again  
[haptic_nv][0487]parse_chipid: aw86224 or aw86225 detected  
[haptic_nv][0442]sw_reset: enter!
```

```
[haptic_nv][0867]haptic_nv_boot_init: haptic_nv driver version v0.2.1.2  
[haptic_nv][0582]parse_chipid: aw86233 detected  
[haptic_nv][0521]sw_reset: enter!  
[haptic_nv][0768]aw8623x_interrupt_setup: enter  
[haptic_nv][0770]aw8623x_interrupt_setup: reg SYSINT=0x00
```

```
[I][haptic_nv]parse_chipid: try to replace i2c addr [(0x58)] to read chip id again  
[I][haptic_nv]parse_chipid: aw86243 detected
```

4 功能和接口

Haptic_NV MCU 兼容驱动中定义了一个函数指针结构体，用户在调用初始化函数 haptic_nv _boot_init 完成开机初始化后，即可通过结构体指针 g_func_haptic_nv 调用函数接口，如下：

```
struct aw_haptic_func
{
    int (*check_qualify)(void);           //芯片量产校验
    int (*get_irq_state)(void);           //获取中断状态
    int (*get_f0)(void);                  //获取马达频率 f0
    int (*offset_cali)(void);             //开机偏移校准
    void (*trig_init)(void);              //trig 初始化
    void (*irq_clear)(void);              //读清中断
    void (*haptic_start)(void);           //开始播放
    void (*play_stop)(void);              //停止播放
    void (*cont_config)(void);            //cont 模式相关寄存器配置
    void (*play_mode)(uint8_t);           //播放模式选择
    void (*ram_init)(aw_bool);            // RAMINIT 使能控制
    void (*misc_para_init)(void);         //开机寄存器初始化配置
    void (*interrupt_setup)(void);        //中断配置
    void (*vbat_mode_config)(uint8_t);    //vbat 模式配置
    void (*protect_config)(uint8_t, uint8_t); //马达保护配置
    void (*calculate_cali_data)(void);     //f0 校准值计算
    void (*set_gain)(uint8_t);            //设置增益
    void (*set_wav_seq)(uint8_t, uint8_t); //设置波形
```

<code>void (*set_wav_loop)(uint8_t, uint8_t);</code>	<code>//设置播放次数</code>
<code>void (*set_rtp_autosin)(uint8_t);</code>	<code>//设置 rtp autosin 使能开关</code>
<code>void (*set_rtp_data)(uint8_t *, uint32_t);</code>	<code>//写 rtp 数据</code>
<code>void (*set_fifo_addr)(void);</code>	<code>//设置 FIFO 地址</code>
<code>void (*get_fifo_addr)(void);</code>	<code>//获取 FIFO 地址</code>
<code>void (*set_ram_data)(uint8_t *, int);</code>	<code>//写 ram 数据</code>
<code>void (*get_ram_data)(uint8_t *, int);</code>	<code>//读 ram 数据</code>
<code>void (*set_ram_addr)(void);</code>	<code>//设置 ram 地址</code>
<code>void (*set_repeat_seq)(uint8_t);</code>	<code>//设置波形重复播放</code>
<code>void (*set_base_addr)(void);</code>	<code>//设置基地址</code>
<code>void (*set_trim_lra)(uint8_t);</code>	<code>//设置 TRIM_LRA 寄存器</code>
<code>void (*set_rtp_aei)(aw_bool);</code>	<code>//设置快空中断</code>
<code>void (*get_vbat)(void);</code>	<code>//电池电压检测</code>
<code>void (*get_reg)(void);</code>	<code>// 获取寄存器数据</code>
<code>void (*get_lra_resistance)(void);</code>	<code>//马达阻抗检测</code>
<code>uint8_t (*get_glb_state)(void);</code>	<code>//获取全局状态寄存器的值</code>
<code>uint8_t (*judge_rtp_going)(void);</code>	<code>//判断是否正在播放 RTP</code>
<code>uint8_t (*rtp_get_fifo_afs)(void);</code>	<code>//判断 FIFO 是否为满</code>
<code>void (*f0_show)(void);</code>	<code>//获取马达真实 F0</code>
<code>void (*ram_show)(void);</code>	<code>//回读显示 RAM 波形数据</code>
<code>void (*cali_show)(void);</code>	<code>//获取校准后的马达 F0</code>
<code>void (*irq_handle)(void);</code>	<code>//中断处理函数</code>

```
void (*get_ram_num)(void);           //获取 ram 波形数量

void (*rtp_vib_work)( uint8_t);      // rtp 播放示例函数

void (*set_hw_irq_status)(uint8_t);  //设置中断状态

uint8_t (*get_hw_irq_status)(void);   //获取中断状态

int (*f0_cali)(void);                //F0 校准

int (*rtp_going)(void);              //rtp 播放

int (*long_vib_work)(uint8_t, uint8_t, uint32_t); //长震播放示例函数

int (*short_vib_work)(uint8_t, uint8_t, uint8_t); //短震播放示例函数

int (*dual_short_vib)(uint8_t, uint8_t, uint8_t, uint8_t, uint8_t, uint8_t); //双马达短震示例函数

int (*dual_long_vib)(uint8_t, uint8_t, uint8_t, uint8_t, uint32_t); //双马达长震示例函数

};
```


5 单马达调用示例

1. 初始化流程：

调用初始化函数 `haptic_nv_boot_init()` 进行 haptic 芯片初始化，如果返回值为 0 则表示初始化成功，只有在初始化完成后才可以使用其他接口；

2. 短振调用示例：

```
g_func->short_vib_work(1, 0x80, 1);
```

3. 长振调用示例：

```
g_func->long_vib_work(4, 0x80, AW_MS_UINT*500);
```

4. RTP 播放示例：

```
g_func->rtp_vib_work(0x80);
```

5. F0 校准：

```
g_func->f0_cali();
```

6. 校准后 F0 读取：

```
g_func->cali_show();
```

7. 真实 F0 读取：

```
g_func->f0_show();
```

8. 马达阻抗检测：

```
g_func->get_lra_resistance();
```

9. 电池电压检测：

```
g_func->get_vbat();
```

6 双马达调用示例

1. 初始化流程：

```
haptic_nv_change_motor(MOTOR_L);           //切换控制左马达  
  
haptic_nv_boot_init();                     //左马达芯片初始化  
  
haptic_nv_change_motor(MOTOR_R);           //切换控制右马达  
  
haptic_nv_boot_init();                     //右马达芯片初始化
```

2. 短振调用示例：

```
g_func_haptic_nv->dual_short_vib(1, 0x80, 1, 1, 0x80, 1);           //双马达同时短振
```

3. 长振调用示例：

```
g_func_haptic_nv->dual_long_vib(4, 0x80, 4, 0x80, AW_MS_UINT*500); //双马达同时长振 0.5s
```

4. F0 校准：

```
haptic_nv_change_motor(MOTOR_L);           //切换控制左马达  
  
g_func_haptic_nv->f0_cali();                //左马达 f0 校准  
  
haptic_nv_change_motor(MOTOR_R);           //切换控制右马达  
  
g_func_haptic_nv->f0_cali();                //右马达 f0 校准
```

5. 校准后 F0 读取：

```
haptic_nv_change_motor(MOTOR_L);           //切换控制左马达  
  
g_func_haptic_nv->cali_show();              //左马达校准后 f0  
  
haptic_nv_change_motor(MOTOR_R);           //切换控制右马达  
  
g_func_haptic_nv->cali_show();              //右马达校准后 f0
```

6. 真实 F0 读取：

```
haptic_nv_change_motor(MOTOR_L);           //切换控制左马达  
  
g_func_haptic_nv->f0_show();                //左马达真实 f0 检测
```

```
haptic_nv_change_motor(MOTOR_R);           //切换控制右马达  
g_func_haptic_nv->f0_show();                //右马达真实 f0 检测
```

7. 马达阻抗检测:

```
haptic_nv_change_motor(MOTOR_L);           //切换控制左马达  
g_func_haptic_nv->get_lra_resistance ();    //左马达阻抗检测  
haptic_nv_change_motor(MOTOR_R);           //切换控制右马达  
g_func_haptic_nv->get_lra_resistance ();    //右马达阻抗检测
```

8. 电池电压检测:

```
haptic_nv_change_motor(MOTOR_L);           //切换控制左马达  
g_func_haptic_nv->get_vbat();               //左马达电压检测  
haptic_nv_change_motor(MOTOR_R);           //切换控制右马达  
g_func_haptic_nv->get_vbat();               //右马达电压检测
```

7 F0 校准说明

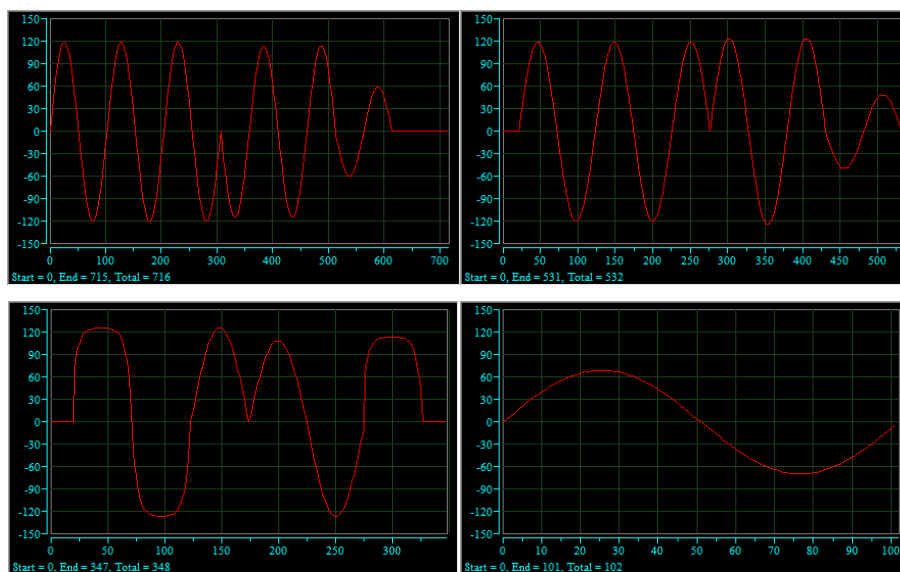
haptic_nv_config.c 中预留了两个保存和获取 F0 校准值的接口，需要客户自行实现:

1. void haptic_nv_set_cali_to_flash(void) //将校准值写入 flash 保存
2. void haptic_nv_get_cali_from_flash(void) //从 flash 读取校准值更新到驱动

如果定义了 AW_F0_CALI_DURING_STARTUP 宏，则默认设备在开机阶段进行 F0 校准，以上两个接口不会使用

8 波形示例

RAM 示例波形如下：



其中 wave1~wave3 为短震波形，wave4 为长震指定波形。

9 注意事项

1. [rtp 播放需配置外部中断 irq_pin,下降沿触发模式](#)：当检测硬件中断触发时，跳转中断函数处理；
2. rtp 模式下，对应 i2c 需单独供 haptic 使用；
3. 驱动需要先调用 haptic_nv_boot_init 函数初始化完成后才可使用 g_func_haptic_nv 指针；
4. 目前双马达驱动不支持 RTP 模式。

版本记录

版本	日期	说明
V1.0	2021-11-22	初版
V1.1	2022-01-27	1. 增加 haptic_nv_config.c 的描述，完善驱动适配步骤
V1.2	2022-06-06	1. 关键信息使用红色标记 2. 添加移植有效性验证 3. 添加水印
V1.3	2022-08-05	1. 添加 aw8623x; 2. 删除 get_first_addr 接口; 3. 删除 AW_LOG_CTRL;
V1.4	2022-08-18	1. 添加双马达
V1.5	2023-02-14	1. 删除多余 h 文件; 2. 文档模板更换;
V1.6	2024-02-05	1. 添加 AW_RTP_AUTOSIN 宏控说明; 2. 添加 aw8624x 支持;

免责声明

此文档中包含的信息被认为是准确、可靠的。但是，上海艾为电子技术股份有限公司（以下简称艾为）对这些信息的准确性或完整性均不作任何明示或暗示的陈述或保证，且对这些信息的使用后果不承担任何责任。

艾为保留在任何时间、没有任何通报的前提下修改本文档中发布的信息，包括但不限于产品资料和规格的权利。客户在下订单前应自行获取最新的相关信息，并验证这些信息是最新且完整的。本文档信息覆盖并取代所有先于此次公布的文档。

艾为的产品没有设计、授权或保证适用于在医疗、军事、飞行器、太空或生命支持设备中使用，或是在可合理地预估艾为产品的故障或失效将导致人身伤害、死亡或严重的财产、环境损害之场合等应用。艾为不接受在上述设备或应用中纳入或使用艾为产品而衍生的任何相关责任，若在此情况下纳入或使用艾为产品，由此而产生的任何风险和责任由客户自身承担。

此文档中对艾为任何产品应用的描述仅用于举例说明，在未经进一步测试和改进的情况下，艾为不对这些应用将会适用于特定用途而作出任何陈述或保证。

所有产品的销售都必须遵守订单确认时艾为提供的商业销售一般条款和条件。

此文档中的任何文字或表述都不能被解释或解读为艾为产品可供承诺的销售要约，也不能被解读为任何专利、版权或其他工业、知识产权等任何许可的授权、转让或暗示。

对于艾为产品技术文档的信息，仅在没有对内容进行任何篡改且带有相关授权、条件、限制和声明的情况下才允许进行复制。艾为对篡改过的文件不承担任何责任或义务。复制第三方的信息可能需要遵守额外的限制条件。

在转售艾为产品或服务时，如果对该产品或服务参数的陈述与艾为标明的参数相比存在差异或虚假成分，则艾为就产品或服务所有明示或暗示的授权将失效，且这是不公平的、欺诈性商业行为。艾为对任何此类陈述均不承担任何责任或义务。